

LeetCode 1044: Longest Duplicate Substring

Frequently Asked In: Oracle, Amazon

© Authored by Arpit Raj, LNMIIT Jaipur | Date: June 30, 2026

1. Problem Statement & Brute Force Analysis

Goal: Find and return the longest substring that occurs at least twice in a given string. If no such substring exists, return an empty string.

The Brute Force Approach

A naive solution would be computationally expensive:

- Generate every possible substring: $O(n^2)$ operations.
- Compare every pair of substrings to check for duplicates: $O(n)$ per comparison.

Total Complexity: $O(n^3)$. This approach guarantees a Time Limit Exceeded (TLE) error for reasonably large string constraints.

2. The Optimal Strategy

1. Monotonicity & Binary Search

Observation: If a duplicate substring of length $L = 5$ exists, then duplicate substrings of lengths 4, 3, 2, and 1 MUST also exist. Conversely, if no duplicate substring of length L exists, it is impossible for a duplicate of length $> L$ to exist.

This monotonic behavior allows us to perform a **Binary Search on the Answer (Length)**:

- If a duplicate of length `mid` exists → Search larger lengths (`low = mid + 1`).

- If NO duplicate of length mid exists → Search smaller lengths ($high = mid - 1$).

2. Polynomial Rolling Hash (Rabin-Karp Algorithm)

To efficiently check if a substring of length mid exists multiple times, we use **Rolling Hashing**. Instead of comparing strings character-by-character $O(L)$, we assign a unique integer to each string window and compare the integers in $O(1)$ time.

When our sliding window moves one step to the right (e.g., from "abcd" to "bcde"), we don't calculate the new hash from scratch. Instead, we update it in $O(1)$:

1. **Remove** the outgoing character's contribution.
2. **Shift** the remaining window by multiplying it by the BASE.
3. **Add** the incoming character's value.

$$Hash = (Hash - (OutChar \times BASE^{L-1})) \times BASE + InChar$$

3. Complexity Summary

- **Binary Search:** Takes $O(\log n)$ steps to find the correct length.
- **Rolling Hash Verification:** Takes $O(n)$ time for each length checked.
- **Overall Time Complexity:** $O(n \log n)$ — significantly optimized from $O(n^3)$.
- **Space Complexity:** $O(n)$ for storing hash mappings to handle potential collisions.

4. C++ Implementation

The code uses a polynomial base of 67 and manages potential hash collisions by falling back to standard string comparison when hashes match.

```
class Solution {
public:

    int check(string s, int k) {
        if (k == 0) return 0;
        int n = s.size();
        const unsigned long long BASE = 67; // Base for polynomial hashing
        unsigned long long hash = 0;
        unsigned long long highestpow = 1;

        for (int i = 0; i < k - 1; i++) {
            highestpow *= BASE; // Max power is BASE^(k-1)
        }

        for (int i = 0; i < k; i++) {
            hash = hash * BASE + (s[i] - 'a' + 1); // First window's hash
        }

        // A map to store indices corresponding to hash values (handles collisions)
        unordered_map<unsigned long long, vector<int>>> mp;

        mp[hash].push_back(0); // First window hash corresponds to 0th index

        for (int j = k; j < n; j++) {
            int outval = s[j - k] - 'a' + 1;
            int inval = s[j] - 'a' + 1;

            hash -= (unsigned long long)outval * highestpow;
            hash *= BASE; // Shift power
            hash += inval;

            int startidx = j - k + 1;

            if (mp.count(hash)) {
                // Hash collision or duplicate check: Compare strings directly
                for (int idx : mp[hash]) {
                    bool same = true;
                    for (int i = 0; i < k; i++) {
                        if (s[idx + i] != s[startidx + i]) {
                            same = false;
                            break;
                        }
                    }
                }
                if (same) return startidx;
            }
        }
    }
};
```

```

        }
    }

    mp[hash].push_back(startidx);
}

return -1;
}

string longestDupSubstring(string s) {
    int n = s.size();
    int low = 0;
    int high = n - 1;
    int bestlen = 0;
    int start = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        int idx = check(s, mid);
        if (idx != -1) {
            bestlen = mid;
            start = idx;
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    if (start == -1) return "";

    return s.substr(start, bestlen);
}
};

```